Хакатон 4.0

Кейс: ИИ-ревьюер кода (DevSecOps) в Git-репозиториях

О компании

Стеллар — технологическая компания, которая формирует будущее цифровых решений







Мы — ведущий разработчик в сферах финтеха, государственного сектора и HR

Наши ключевые направления:

1. Развитие платформенных продуктов
2. Внедрение искусственного интеллекта
3. Создание надёжных систем кибербезопасности

О компании

В основе Стеллар — сильная инженерная культура и продуктовый подход.
Мы создаём как кастомные решения, так и собственные технологические продукты:

Интеллектуальная система контроля качества кода на базе AI/ML

QUAL DEVCODE

Платформа для проактивной защиты данных и предотвращения киберугроз

CYBER[code]SEC

Описание кейса

Название

Автоматизированный ИИ-пользователь для проверки Pull/Merge Requests на уязвимости

Задача

Разработать автоматизированную DevSecOps-систему для ускорения ревью кода и предотвращения попадания уязвимостей в основную ветку разработки

Краткое описание

Требуется разработать микросервис (бота), который интегрируется в GitLab, GitHub или Bitbucket

При создании нового Pull Request бот автоматически

- читает изменения кода (diff)
- анализирует их с помощью LLM на предмет проблем с безопасностью
- оставляет комментарии (code review) прямо в строках кода

Дополнительные условия

Стек работы

Языки программирования:

- Python
- Go
- Node.js

Технологии и техники:

- Webhooks API (GitHub/GitLab API)
- Docker
- REST API
- LLM API (OpenAI / Claude / локальные)

Кто нужен в команде

- **Бэкенд-разработчики**
  Настройка API и Webhooks
- **DevOps-инженеры**
  Настройка пайплайнов и CI/CD
- **Специалисты по ИБ / AI-инженеры**
  Написание качественных системных промптов для поиска уязвимостей

Дополнительные условия

Требования

Интеграция

- Портал должен позволять администратору привязать репозиторий (через токены доступа)
- Бот должен выступать от лица отдельного сервисного аккаунта

Логика анализа

- При триггере PR (Pull Request) система выкачивает git diff
- Формируется промпт, в который передаются изменения, а также строгий контекст
- Анализатор должен избегать ложноположительных срабатываний (False Positives)

План выполнения (на 24 часа)

Часы 1-5

Настройка тестового репозитория. Написание эндпоинта для приема Webhook

Часы 5-12

Написание скрипта взаимодействия с LLM. Подбор системного промпта

Часы 12-18

Реализация обратной отправки комментариев через API репозитория

Часы 18-24

Обработка краевых случаев, сборка в Docker, подготовка презентации

Образ результата

Особенность реализации

Бот должен работать как реальный пользователь репозитория:

01

Реагировать
на события (webhooks)

02

Анализировать только
изменённый код
(чтобы экономить токены
и время)

03

Публиковать советы
по исправлению кода
непосредственно в интерфейсе
системы контроля версий

Критерии оценки

Каждый пункт оценивается по шкале от 0 до 3

Приоритет	Содержимое группы приоритета	Вес (w)
1	Сервис успешно принимает Webhook о создании Pull/Merge Request из репозитория	**6**
1	Сервис извлекает только измененные строки кода (diff) и отправляет их на анализ в LLM	**6**
1	Система распознаёт базовые уязвимости (SQL-инъекции, захардкоженные пароли/токены, XSS)	**6**
1	Бот автоматически оставляет комментарий к Pull Request с указанием проблемной строки и сути уязвимости	**6**
2	ИИ предлагает конкретный кусок кода (сниппет) для исправления найденной уязвимости	**3**
2	Бот фильтрует файлы, не требующие проверки (например, README.md, .gitignore, картинки)	**3**
3	Система позволяет пользователю ответить боту в комментариях для уточнения, и бот поддерживает диалог	**1**
3	Наличие возможности заблокировать слияние ветки (Merge), если ИИ нашёл критическую уязвимость	**1**

Хакатон 4.0

Желаем удачи!